Super-Convergence

Сравнение CLR и step LR на CIFAR-10

Отчёт по статье Leslie N. Smith и Nicholay Topin, Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates [1].

1. Что проверяли

Авторы статьи показывают, что один цикл с большим LR позволяет обучить модель быстрее и сохранить accuracy. Например, на рисунке 1a обычный режим даёт 91,2% примерно за 80 000 шагов, а super-convergence - 92,4% за 10 000. Авторы объясняют это в том числе регуляризацией за счёт большого LR.

Мы взяли короткий запуск из таблицы 6 и сравнили его со step LR. Хотели проверить, какой режим раньше наберёт 90% accuracy на CIFAR-10.

2. Как запускали

Обучали ResNet-56 с нуля на одной NVIDIA L4: 50 000 изображений для обучения и 10 000 для теста. В обоих запусках использовали seed 0, одинаковые начальные веса, порядок данных и случайные горизонтальные отражения. Из изображений вычитали среднее по обучающей выборке.

Для CLR взяли schedule из статьи: LR растёт от 0,1 до 3 за первую тысячу шагов, затем снижается обратно к 0,1. Всего 2000 шагов, или 40 эпох. Для сравнения запустили step LR на 4000 шагов: LR 0,1, после 2000 шагов - 0,01, после 3000 - 0,001.

Batch size - 1000, microbatch - 125, momentum - 0,9, weight decay - 0,0001, BN fraction - 0,80. В PyTorch перенесли архитектуру, правило обновления SGD и накопление статистик BN из кода авторов [2]. Для оценки использовали статистики BN, накопленные во время обучения.

Accuracy измеряли каждые 100 шагов. Отмечали первый замер с accuracy ≥ 90%. Этот порог и длину ступенчатого запуска выбрали после просмотра CLR, но до запуска сравнения.

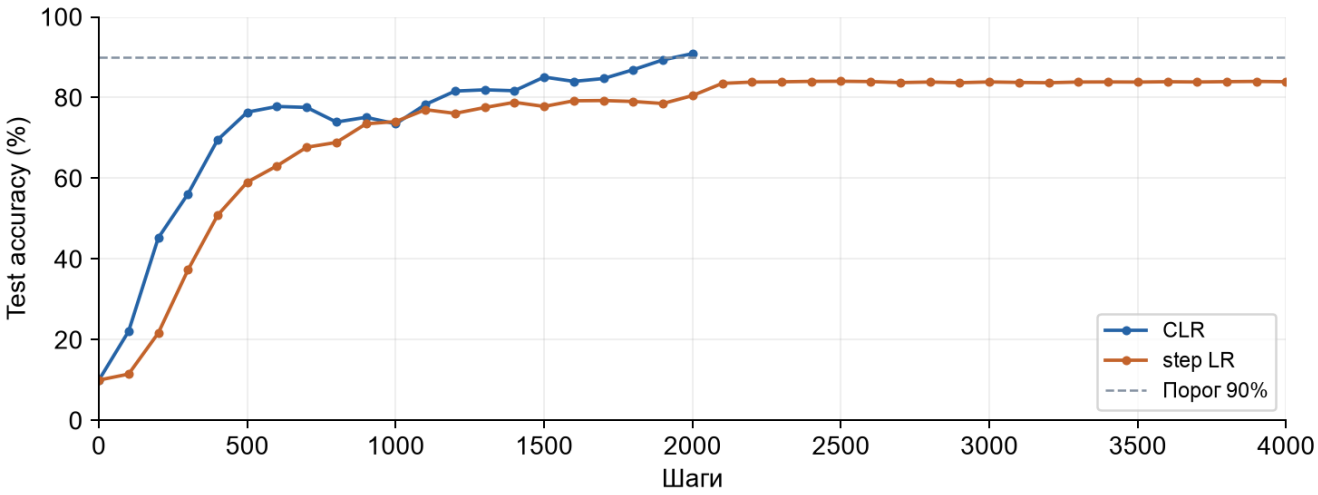
3. Что получилось

Режим	Шаги	Test accuracy	Время обучения
CLR	2000	90,90%	11 мин 48 с
step LR	4000	83,93%	23 мин 54 с

В нашем эксперименте CLR дал 90,90% accuracy за 2000 шагов, step LR - 83,93% за 4000. Разница составила 6,97 процентного пункта.

В таблице 6 статьи для CLR указано 89,7% за 2000 шагов, у нас - 90,90% за те же 2000 шагов. Время в таблице включает подачу данных и шаги обучения.

4. Как менялась accuracy



Каждая точка - замер на всех 10 000 test-изображениях. Линии напрямую соединяют соседние замеры. Порог 90% проверяли в этих точках.

5. Как повторить запуск

Мы использовали Python 3.12.15, PyTorch 2.11.0+cu130 и NVIDIA L4. Команды из корня проекта:

```
python -m pip install -r requirements.txt
export CUBLAS_WORKSPACE_CONFIG=:4096:8
python train.py --method paper --seed 0 --device cuda \
  --workers 4 --threads 4 --deterministic
python train.py --method step --seed 0 --device cuda \
  --workers 4 --threads 4 --deterministic
python plot.py runs/paper runs/step --output results/repeat
```

Весь код обучения находится в train.py, графики строит plot.py. Метрики и настройки сохранены в results/gcp-l4-table6-2000-20261010/ и results/gcp-l4-speed-4000-20261010/. Веса и данные создаются при запуске.

Источники

[1] Smith, Topin. Super-Convergence, arXiv:1708.07120v3. Рисунок 1a и таблица 6.

[2] Код авторов, commit ecc561594b. x.sh, clrsolver.prototxt и Resnet56Cifar.prototxt.